1 Setup and Assumptions

1.1 Simulated system

The simulator models a 32-bit virtual address space with 4 KB pages, so an address splits into a 20-bit virtual page number (VPN) and a 12-bit offset, giving 2^{20} virtual pages. Only the lower 32 bits of each trace address are used (the traces contain addresses wider than 32 bits). The TLB has 128 entries, is fully associative with LRU replacement, and is empty at the start of every run. Instruction/data and read/write flags in the traces are not used.

1.2 Cost model

Every trace line is one memory access of the benchmark. Let w be the number of memory accesses needed to walk the page table ($w = 1$ for the single-level table, $w = 2$ for the two-level table). The cycles charged per access are:

Case	Cycles
No TLB	$200\,w + 200$
TLB hit	$1 + 200$
TLB miss	$1 + 200\,w + 200$
Page replacement (extra, Part IV only)	+20,000,000

The final 200 cycles is the access to the data itself. A page fault into a free frame costs the same as any page-table walk and no disk time, since the first pages come for free; disk time is charged only when a resident page has to be replaced. We do not model where the page table itself is stored in physical memory: page-table pages do not occupy frames, and only the cost of walking them is charged.

1.3 Frame management scheme

- **Parts I–III** (more frames than pages): frames are handed out sequentially, $0, 1, 2, \ldots$, in the order pages are first touched, and are never freed. The number of frames used is therefore the number of distinct pages touched.
- **Part IV** (limited frames): frames are initially handed out the same way. Once all frames are in use, a page fault evicts the least recently used page. Recency is kept in a doubly linked list over the frames; every access (TLB hit or not) moves the accessed page's frame to the most-recently-used end, and the victim is the frame at the least-recently-used end. On eviction the simulator removes the victim from the page table, invalidates its TLB entry (so the TLB can never return a translation to a frame that has been reused), charges the disk cost, and gives the freed frame to the faulting page.

1.4 Page tables

The *single-level* table is an array of 2^{20} entries indexed by the VPN. The *two-level* table splits the VPN into a 10-bit directory index and a 10-bit second-level index. Second-level tables (1024 entries each) are allocated only when the first page they cover is mapped, and are freed when their last valid entry is removed. Each second-level table therefore covers 1024 pages (a 4 MB region of the address space).

1.5 Statistics

A. Frames used the peak number of frames in use.

B. Hot pages the number of accesses to every virtual page, from which the cumulative access share of the hottest pages is computed.

C. Cycles total cycles over the trace, using the cost model above.

D. Page table utilization number of distinct pages accessed divided by the number of page table entries allocated. For the single-level table the denominator is 2^{20}. For the two-level table it is 1024× (number of live second-level tables); the fixed 1024-entry top-level directory is *not* counted.

E. Hash $H \leftarrow (H \oplus \mathit{frame}) \times 1099511628211 \pmod{2^{64}}$, applied once per access with $H = 0$ initially.

1.6 Environment

All numbers come from one run of each program on an Intel Core Ultra 7 258V (8 logical CPUs, 30 GB RAM), compiled with g++ 13.3.0 using `-std=c++17 -O2`. The ten benchmark traces contain between 11.5 and 12.9 million accesses each. Traces are streamed from disk in 1 MB chunks and parsed by hand.

2 Part I: Single-Level Page Table

Table 1 shows the results with a single-level table, no TLB, and enough frames that no page is ever evicted.

Table 1: Part I results (single-level page table, no TLB). "Top-10% share" is the fraction of all accesses that go to the most-accessed 10% of the pages the benchmark touches.

Benchmark	Accesses (M)	Frames used	PT util. (%)	Top-10% share (%)	Cycles (10^9)
blender	12.37	1551	0.1479	99.2	4.948
fotonik3d	12.89	1602	0.1528	91.3	5.156
gcc	12.47	990	0.0944	82.5	4.989
imagick	11.55	96	0.0092	89.1	4.620
lbm	12.84	2131	0.2032	94.1	5.137
mcf	12.47	1597	0.1523	97.9	4.990
perlbench	12.57	630	0.0601	89.3	5.027
x264	11.72	234	0.0223	90.0	4.689
xalancbmk	11.97	14325	1.3661	95.2	4.789
xz	11.85	8037	0.7665	99.5	4.741

Frames used (A). Each benchmark touches only 96 to 14 325 distinct pages out of the $2^{20} =$ 1,048,576 available, so that many frames are used.

Hot pages (B). Figure 1 plots, for each benchmark, the cumulative share of accesses covered by the hottest pages (pages sorted by access count, hottest first). In every benchmark the hottest 10% of touched pages account for 82.5%–99.5% of all accesses (Table 1), which supports the hypothesis that a small fraction of pages sees a large fraction of the accesses. The fractions in this figure are of the pages each benchmark *touches*, not of the whole address space.

Page table utilization (D). Only 0.009%–1.37% of the entries of the single-level table are ever accessed. The table always has 2^{20} entries allocated, whatever the benchmark touches.

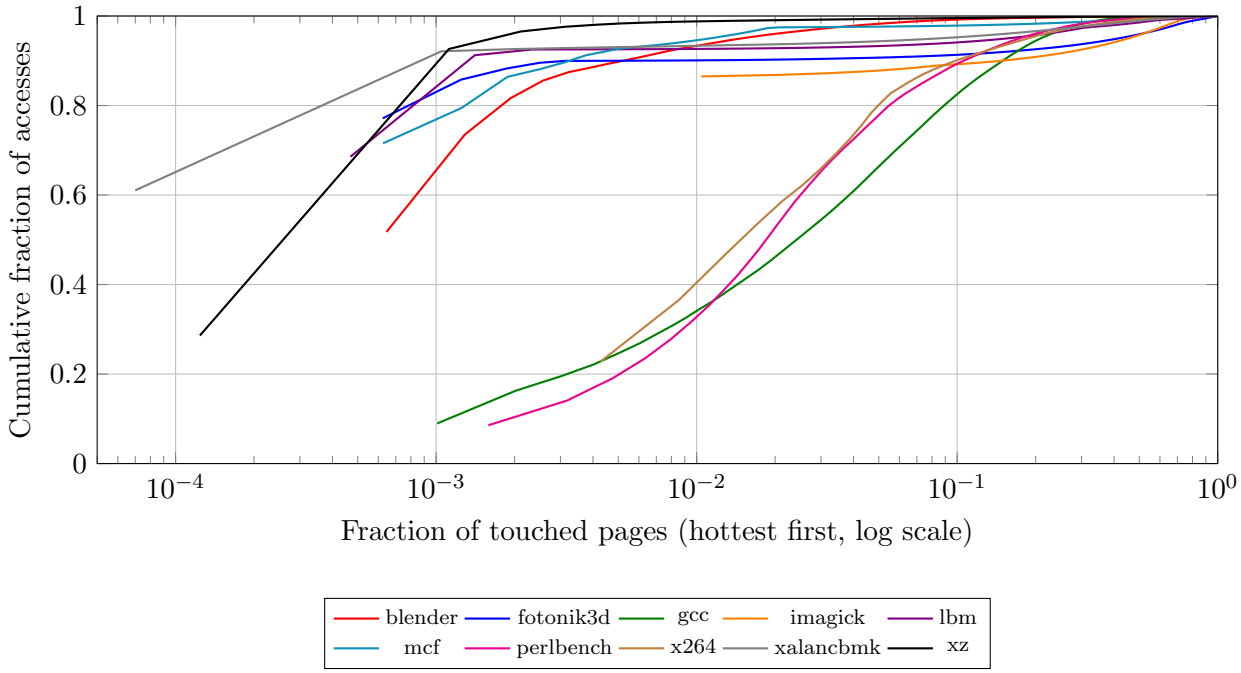


Figure 1: Cumulative access share of the hottest pages for each benchmark (Part I). Each curve is sampled at log-spaced points.

Cycles (C) and hash (E). With no TLB, every access costs 400 cycles (one table access plus the data access), so total cycles are exactly 400× accesses (4.620–5.156 billion cycles). The hashes are listed in `results/part_i.csv`.

3 Part II: Two-Level Page Table

Table 2: Part II results (two-level page table, no TLB), with the single-level numbers for comparison. "L2 tables" is the number of second-level tables allocated. The frames used and hashes are identical to Part I.

Benchmark	L2 tables	Entries alloc. (2L)	Util. 2L (%)	Util. 1L (%)	Cycles 2L (10^9)	Cycles 1L (10^9)
blender	47	48 128	3.22	0.1479	7.422	4.948
fotonik3d	14	14 336	11.17	0.1528	7.734	5.156
gcc	70	71 680	1.38	0.0944	7.484	4.989
imagick	5	5 120	1.88	0.0092	6.929	4.620
lbm	6	6 144	34.68	0.2032	7.706	5.137
mcf	13	13 312	12.00	0.1523	7.485	4.990
perlbench	48	49 152	1.28	0.0601	7.541	5.027
x264	11	11 264	2.08	0.0223	7.033	4.689
xalancbmk	54	55 296	25.91	1.3661	7.183	4.789
xz	43	44 032	18.25	0.7665	7.111	4.741

Hash. The hash computed with the two-level table is identical to the one from Part I for all ten benchmarks. This is expected: the hash depends only on the sequence of frame numbers, the frame allocator is the same in both simulators, and the page table type does not affect which frame a page gets.

Space. The number of pages touched is the same for both table types, so the difference in utilization (Table 2) comes entirely from the denominator. The two-level table allocates only

the second-level tables that cover touched pages, namely 5 to 70 tables per benchmark (one per distinct 4 MB region touched, since nothing is evicted), against the fixed 2^{20} entries of the single-level table. Its utilization is 1.28%–34.7%, compared with 0.009%–1.37% for the single-level table. The top-level directory (1024 entries) is a fixed cost that is not included in these figures.

Time. Each walk of the two-level table needs one more memory access, so every access costs 600 cycles instead of 400, and the total cycles are exactly 1.5 times the single-level total for every benchmark. The two-level table thus trades extra cycles for less allocated page-table space.

4 Part III: Adding a TLB

Table 3: Part III results (128-entry fully associative LRU TLB). Speedup is the Part I/II cycle count (no TLB) divided by the cycle count with the TLB, for the same page table type. The TLB hit rate is the same for both page table types.

Benchmark	TLB hit rate (%)	Cycles 1L (10^9)	Cycles 2L (10^9)	Speedup 1L	Speedup 2L
blender	99.9398	2.488	2.489	1.99	2.98
fotonik3d	99.9867	2.591	2.592	1.99	2.98
gcc	99.8874	2.510	2.513	1.99	2.98
imagick	99.9992	2.321	2.321	1.99	2.99
lbm	99.9789	2.582	2.583	1.99	2.98
mcf	99.4421	2.521	2.535	1.98	2.95
perlbench	99.5484	2.538	2.549	1.98	2.96
x264	99.9659	2.357	2.358	1.99	2.98
xalancbmk	99.8673	2.410	2.413	1.99	2.98
xz	99.7669	2.388	2.393	1.99	2.97

Hit rate (A). The TLB hit rate is 99.44%–99.9992% across the benchmarks (Table 3); the lowest is mcf. The rate is identical for the single-level and two-level tables, because the TLB logic does not depend on the page table type: both simulators present it with the same stream of virtual page numbers.

Cycles (B). With the TLB, total cycles drop to 2.321–2.591 billion. This is a speedup of 1.98–1.99 times for the single-level table and 2.95–2.99 times for the two-level table over the no-TLB runs. The two-level table benefits more because a TLB hit skips the entire page-table walk, and the walk is more expensive for the two-level table. With the TLB, the difference between the two table types shrinks to 200 cycles per TLB miss (one extra memory access per walk), which is a small fraction of the total.

5 Part IV: Limited Physical Memory

The traces touch at most 14 325 pages, which is less than $2^{14} = 16384$ frames, i.e. less than 1/64 of the virtual address space. Physical memory sizes of 1/2, 1/4, 1/8, 1/16 and 1/32 of the virtual address space therefore give exactly the same result as 1/2: nothing is ever evicted. We ran physical memory sizes of 1/262144, 1/65536, 1/16384, 1/4096, 1/1024, 1/256, 1/64 and 1/2 of the virtual address space (2^2, 2^4, 2^6, 2^8, 2^{10}, 2^{12}, 2^{14} and 2^{19} frames). All runs use the TLB. A page fault is any first load of a page into memory, including loads into a free frame; an eviction is a fault that has to replace a resident page, and only evictions are charged the 20 million cycle disk cost.

Table 4: Part IV: page faults per million accesses for each physical memory size (columns are numbers of frames). The fault counts are identical for the single-level and two-level tables. The 16384-frame column equals the compulsory (first-touch) faults.

Benchmark	4	16	64	256	1024	16384
blender	32 489	7 382	3 699	381	151	125
fotonik3d	139 129	3 103	133	133	133	124
gcc	71 887	19 926	3 199	400	79	79
imagick	641	581	568	8	8	8
lbm	37 761	523	211	211	211	166
mcf	80 078	16 953	7 046	3 253	131	128
perlbench	91 255	29 318	9 100	1 620	50	50
x264	14 646	1 631	539	20	20	20
xalancbmk	46 800	2 082	1 381	1 252	1 251	1 197
xz	42 434	10 963	2 779	2 029	1 310	678

When replacement starts (A). In all 160 runs, a benchmark has zero evictions exactly when the number of frames is at least the number of pages it touches (Table 1). Once memory is smaller than that, the number of faults minus the number of evictions equals the number of frames, i.e. memory fills completely before the first eviction.

Cycles (B). Wherever any eviction happens, the disk cost dominates: it accounts for at least 82.9% of the total cycles in every run with at least one eviction. As a result, total cycles rise by orders of magnitude as soon as memory drops below a benchmark's footprint (Figure 3).

Page table type. In every one of the 80 (benchmark, size) pairs, the single-level and two-level simulators have identical fault counts, eviction counts and TLB hit rates; only the cycle counts differ, by the extra walk access. Replacement is decided by the frame manager and does not depend on the page table.

6 Part V: Simulator Speed

Table 5 gives the wall-clock time of the complete simulator (two-level page table with TLB, as in Part III) for each benchmark. The time is measured with a monotonic clock around constructing the simulator, reading the trace and simulating every access; it includes reading the trace from disk and excludes printing. Each figure is from one run, so it is subject to run-to-run noise. The ten benchmarks together take 2.5 s.

Table 5: Simulator time per benchmark (two-level page table with TLB, `-O2`).

Benchmark	Accesses (M)	Time (s)	M accesses/s
blender	12.37	0.211	58.6
fotonik3d	12.89	0.180	71.6
gcc	12.47	0.369	33.8
imagick	11.55	0.127	90.9
lbm	12.84	0.167	76.9
mcf	12.47	0.329	37.9
perlbench	12.57	0.370	34.0
x264	11.72	0.348	33.7
xalancbmk	11.97	0.175	68.4
xz	11.85	0.261	45.4

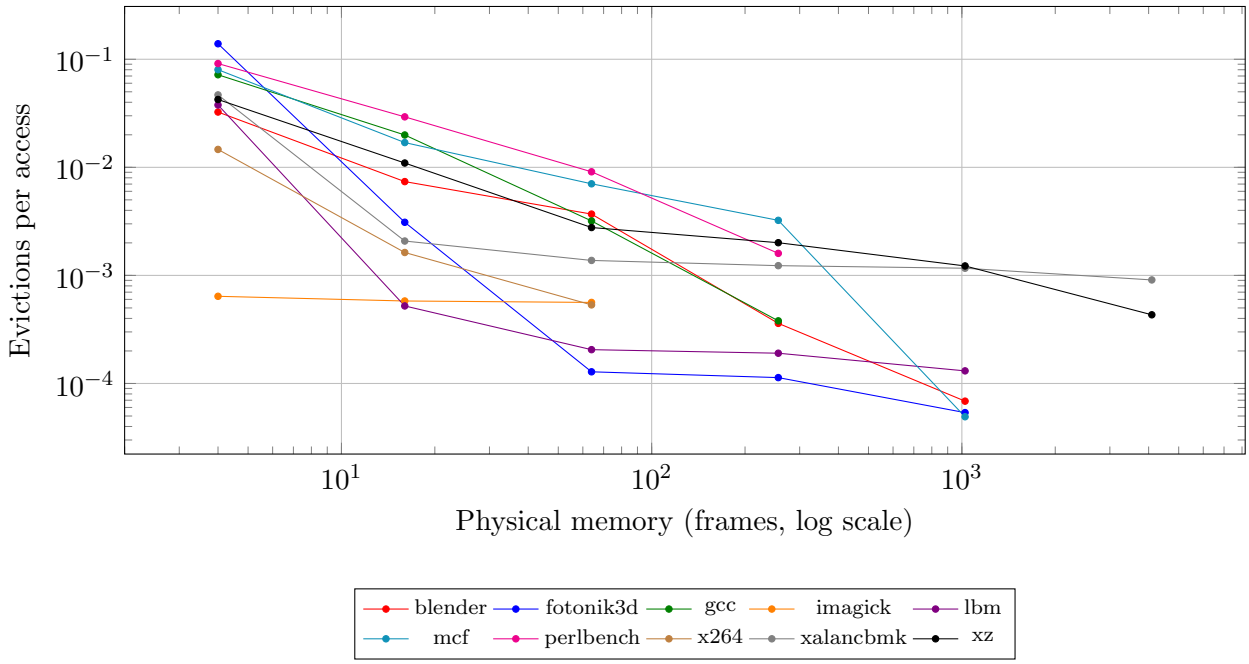


Figure 2: Evictions per access versus physical memory size (two-level table; identical for the single-level table). Runs with zero evictions cannot be drawn on a logarithmic axis and are omitted, so each curve starts at the first memory size where that benchmark evicts a page.

7 Notes and Limitations

- Page table utilization for the two-level table excludes the fixed 1024-entry directory. It is only reported for Parts I–III; in Part IV second-level tables can be freed by evictions, so the value at the end of a run is not comparable.
- "Page fault rate" in Part IV is faults per access including compulsory first-touch faults. The eviction rate (replacements per access) is also in `results/part_iv.csv`.
- Page-table storage is not modelled as using physical frames, so it does not compete with the benchmark's pages for memory in Part IV.
- Timing figures come from a single run on one machine.

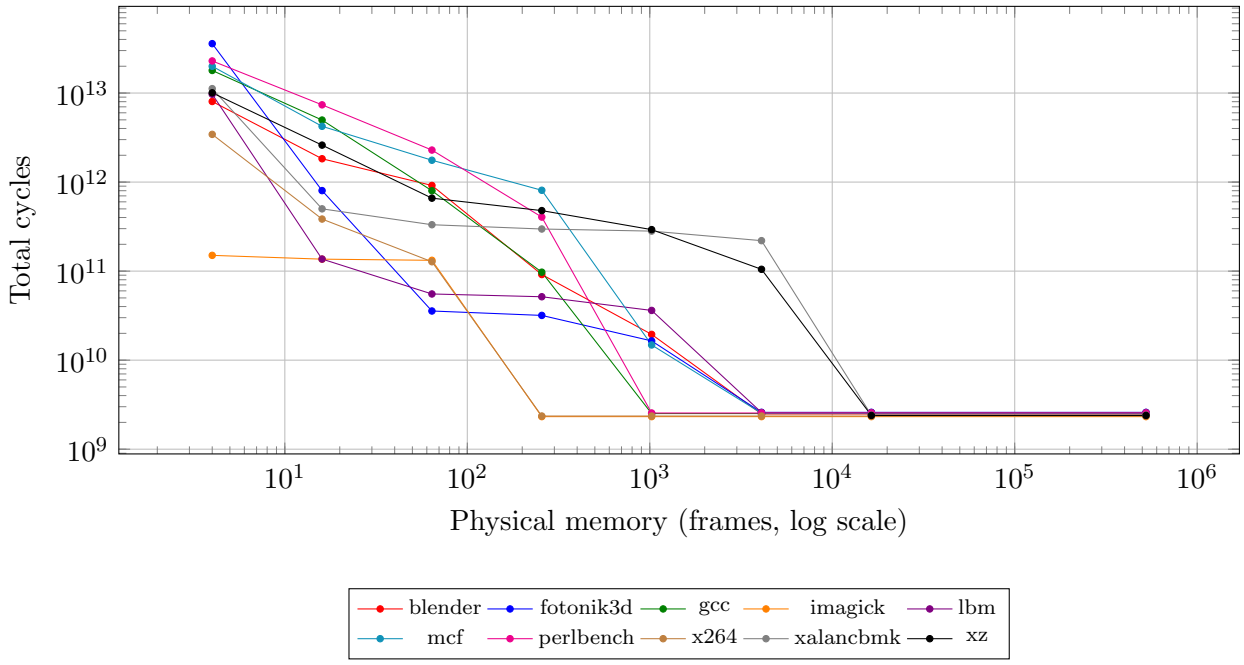


Figure 3: Total cycles versus physical memory size (two-level table with TLB).